

Task 6: Reflection Report ShopEasy — SE3004 Software Testing

Question 1: Which bugs did each technique reveal in the starter code?

The specification-based tests in Task 1 immediately revealed that PriceCalculator was too permissive: it accepted negative discount rates and rates above 100 without validation. Those inputs produced numeric results, but they were not meaningful business values — the code was mathematically executable but semantically wrong.

Structural testing in Task 2 achieved 91% branch coverage on ShoppingCart, but PIT mutation testing proved that coverage alone was not enough. One mutant showed that Product lacked a boundary test for negative unitPrice. Another highlighted that OrderProcessor had untested null-dependency checks. This demonstrated the difference between "code executed" and "behavior really protected."

Design by contract in Task 3 uncovered a consistency problem between implementation and tests. Once precondition assertions were added to PriceCalculator, two Task 1 tests failed because they expected numeric results from invalid inputs. This showed that those tests had been documenting accidental behavior rather than correct behavior — a gap in the starter code's specification, not just an implementation bug.

Property-based testing in Task 4 surfaced a modeling issue: cart commutativity was only valid after accounting for the fact that ShoppingCart merges items with the same product id. This was not a code defect but a defect in the initial property assumption, which is exactly what property-based testing is designed to catch.

Task 5 with mocks enabled direct verification of OrderProcessor's control flow. The strongest check was `verify(..., never()) on charge()`, which confirmed that payment is never attempted when inventory is unavailable — a guarantee that would be difficult to assert without isolation.

Question 2: Which technique was most effective per unit of effort?

Design by contract was the most effective technique per unit of effort. A small number of assert statements immediately clarified valid and invalid inputs, caught mismatches between tests and implementation, and made every other test more trustworthy. Mocks were also high value for verifying OrderProcessor's flow, but contracts improved the shape of the code itself and had unusually high leverage for the amount of work required.

**Question 3: Based on the testing pyramid, how would you classify this test suite?
What is missing?**

The suite is heavily weighted toward unit tests, which is appropriate at this stage. It is strong at the base of the pyramid but weak near the middle and top. What is missing is integration coverage across collaborating components — for example, a test using the actual `OrderProcessor` with a concrete inventory implementation. The suite also lacks end-to-end coverage of the full checkout flow and data-driven regression tests around the boundaries that mutation testing identified, particularly for `Product` construction and cart merge behavior.

Question 4: If you had one more week, what would you test next and why?

With one more week, I would add integration tests around the checkout workflow and mutation-killing boundary tests. The current suite reasons well at the unit level, but the next most valuable risk reduction would come from verifying how classes work together under realistic conditions. I would focus on an integration-style checkout test covering `ShoppingCart`, `OrderProcessor`, inventory, and payment in both success and failure paths. After that, I would add targeted tests for the previously surviving mutant areas — especially `Product` constructor boundaries and cart cases with repeated product ids — to move the suite toward a more balanced testing pyramid.